



DevOps Foundation

Complete Beginner Guide

Simple Words • Real Examples • Hands-On Practice

■ Linux

■ Networking

■ Git

■■ Nginx

■ Bash



Author



Er. Meet Kanteliya

■ Table of Contents

■ CHAPTER 1: Linux

- 1.1 File Permissions & chmod
- 1.2 Users and Groups
- 1.3 Process Management — ps, top
- 1.4 Logs & Debugging
- 1.5 SSH Setup
- 1.6 Basic Bash Scripting

■ CHAPTER 2: Networking

- 2.1 HTTP vs HTTPS
- 2.2 How DNS Works
- 2.3 Browser to Server Journey
- 2.4 Ports & Firewalls
- 2.5 Reverse Proxy with Nginx
- 2.6 Load Balancer Concept

■ CHAPTER 3: Git

- 3.1 Branching Strategy
- 3.2 Pull Request Flow
- 3.3 Merge Conflicts

■ ■ MINI PROJECT: Deploy a Simple App

■ Quick Reference Cheatsheet

Chapter 1: Linux

Linux is the operating system that runs most servers in the world. When you work in DevOps, almost everything happens on Linux. Let's start from the basics.

1.1 File Permissions & chmod

Every file in Linux has **permissions**. Permissions decide **who can read, write, or run** a file.

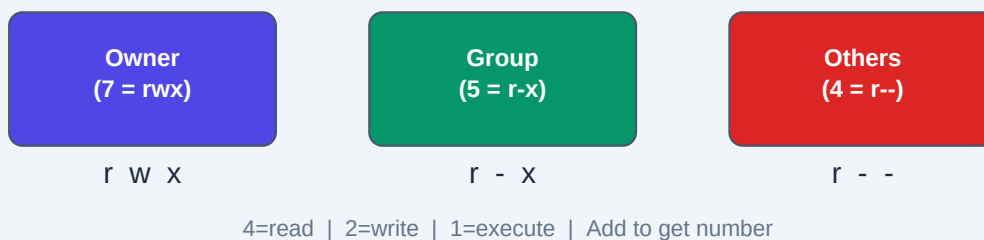
What are permissions?

- **r** = read — you can look at the file
- **w** = write — you can change the file
- **x** = execute — you can run the file as a program

When you run `ls -l` you see:

```
$ ls -l
-rwxr-xr-- 1 alice devteam 1234 app.sh
#  ^^^  ^^^  ^^^
#  |    |    +-- others: r-- (read only)
#  |    +----- group:  r-x (read + execute)
#  +----- owner: rwx (read + write + execute)
```

File Permission — chmod 754



The chmod command

```
# Give owner full access, group read+exec, others read only
chmod 754 app.sh

# Make a script runnable
chmod +x script.sh

# Remove write from others
```

```
chmod o-w file.txt
```

■ Permission Numbers

4 = read | 2 = write | 1 = execute

Add them: 7=rwx | 6=rw- | 5=r-x | 4=r--

chmod 755 → owner=7(rwx), group=5(r-x), others=5(r-x)

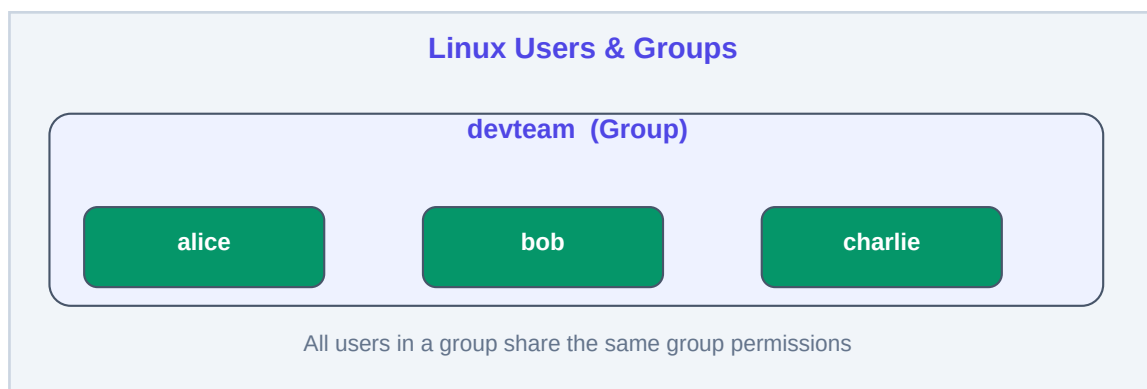
■ **Real World:** At a company, teammates should read files but not delete them. Use `chmod 744` — you can do everything, they can only read.

⇒ ■ PRACTICE EXERCISE

- Create a file: `touch myfile.txt`
- Check permissions: `ls -l myfile.txt`
- Make executable: `chmod +x myfile.txt`
- Owner only: `chmod 700 myfile.txt`
- Check again: `ls -l myfile.txt`

1.2 Users and Groups

Linux can have many users. Each person gets their own account. Groups let you manage many users at once.



User Commands

```
whoami          # who am I?
cat /etc/passwd  # list all users
sudo useradd -m bob  # create user bob
```

```
sudo passwd bob          # set password for bob
sudo userdel bob         # delete user bob
```

Group Commands

```
sudo groupadd devteam    # create group
sudo usermod -aG devteam bob # add bob to group
groups bob               # see bob's groups
cat /etc/group           # list all groups
```

■ **Note:** The root user is the super admin. It can do anything. Be careful!

■ **Tip:** sudo means "run as root for just this one command". Always safer than logging in as root.

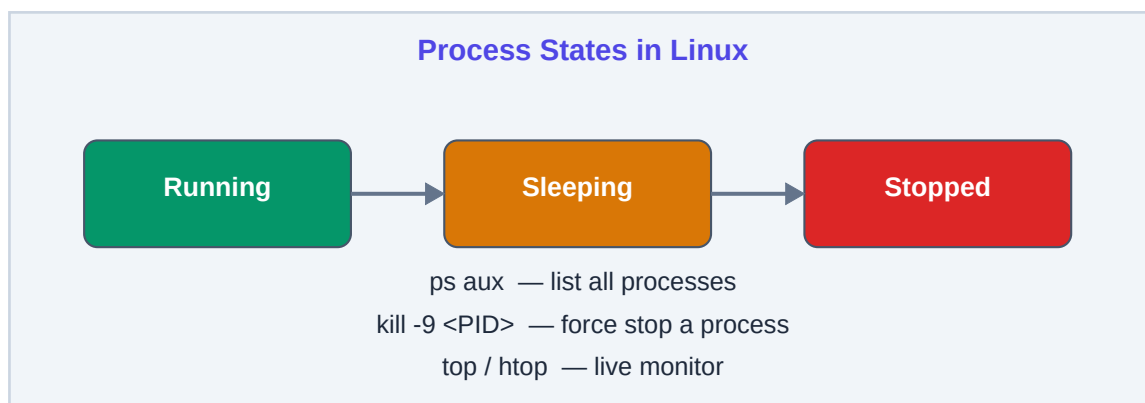
■ **Real World:** All developers are in "devteam" group. The code folder has group rw-. Everyone in devteam can read and write code. Outsiders cannot touch it.

⇒ ■ PRACTICE EXERCISE

- Run: whoami and id
- Create a group: sudo groupadd mygroup
- Add yourself: sudo usermod -aG mygroup \$USER
- Check: groups \$USER

1.3 Process Management — ps, top

A **process** is any program that is running. Your web server, database, and SSH connection are all processes.



View Processes

```
ps aux          # list ALL running processes
```

```
ps aux | grep nginx      # find a specific process

top                      # live monitor (like Task Manager)

htop                    # better live monitor
```

Reading ps aux output:

USER	PID	%CPU	%MEM	COMMAND
root	1	0.0	0.1	/sbin/init
alice	1234	5.2	2.4	python app.py
www	1500	1.1	1.8	nginx: worker

- **PID** = Process ID (unique number for each process)
- **%CPU** = how much CPU it is using
- **%MEM** = how much memory it is using

Control Processes

```
kill 1234                # ask process to stop

kill -9 1234             # force stop (immediately)

pkill nginx              # stop by name

python app.py &          # run in background

jobs                     # see background jobs
```

■ **Real World:** Your app is using 100% CPU. Run "`ps aux | grep python`" to find the PID, then "`kill -9 1234`" to stop it.

⇒ ■ PRACTICE EXERCISE

- Run: `ps aux` — look at all processes
- Run: `top` — watch live (press q to quit)
- Run: `sleep 100` & then `ps aux | grep sleep`
- Kill it: `kill`

1.4 Logs and Debugging

Logs are files where the system writes messages. When something breaks, logs tell you what went wrong and when.

Log File	What it stores
----------	----------------

/var/log/syslog	General system messages
/var/log/auth.log	Login and SSH access
/var/log/nginx/access.log	All web requests
/var/log/nginx/error.log	Nginx errors
/var/log/dpkg.log	Package installs

Log Commands

```
cat /var/log/syslog          # read full log
tail -n 50 /var/log/syslog  # last 50 lines
tail -f /var/log/nginx/access.log # watch LIVE
grep "error" /var/log/syslog # find errors
grep -i "502" /var/log/nginx/error.log # find 502 errors
```

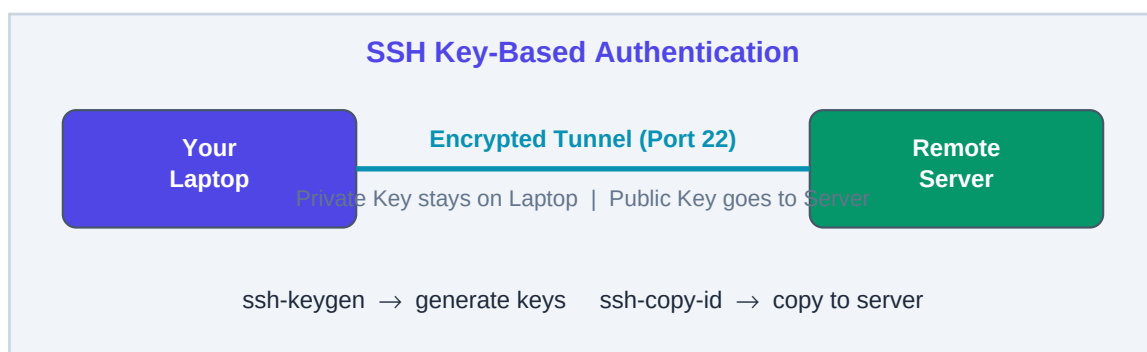
■ **Real World:** App crashed at 3am. Run `"tail -n 200 /var/log/nginx/error.log | grep error"` — you see **"502 Bad Gateway"**. Your backend stopped. Restart it!

⇒ ■ PRACTICE EXERCISE

- Run: `tail -f /var/log/syslog` (Ctrl+C to stop)
- In another terminal: `sudo apt update`
- Watch new log lines appear
- Run: `grep "sshd" /var/log/auth.log`

1.5 SSH Setup

SSH (Secure Shell) lets you log into another computer securely over the internet. This is how DevOps engineers connect to servers.



Basic SSH

```
ssh alice@192.168.1.100      # connect to server

ssh -p 2222 alice@server.com # different port

scp file.txt alice@server:/home/alice/  # upload file

scp alice@server:/home/alice/report.txt . # download file
```

SSH Key Setup (No Password!)

```
# Step 1: Generate key pair (on YOUR laptop)
ssh-keygen -t ed25519 -C "alice@company.com"

# Step 2: Copy public key to server
ssh-copy-id alice@192.168.1.100

# Step 3: Login without password
ssh alice@192.168.1.100

# Keys are here:

# ~/.ssh/id_ed25519      ← PRIVATE key (NEVER share)
# ~/.ssh/id_ed25519.pub ← PUBLIC key (safe to share)
```

■ **Note:** Never share your private key. The public key is safe to put on servers.

■ **Tip:** Create ~/.ssh/config for shortcuts: Host myserver → HostName 192.168.1.100

⇒ ■ PRACTICE EXERCISE

- Generate keys: `ssh-keygen -t ed25519`
- View public key: `cat ~/.ssh/id_ed25519.pub`
- If you have a server: `ssh-copy-id user@server`
- Create SSH config file for shortcuts

1.6 Basic Bash Scripting

A **bash script** is a file with many commands. Instead of typing 10 commands every time, write them once and run the file.

Your First Script

```
#!/bin/bash

echo "Hello! I am a bash script"
```



```
echo "Today: $(date)"

echo "You are: $(whoami)"

# Save as hello.sh, then:

chmod +x hello.sh

./hello.sh
```

Variables

```
#!/bin/bash

name="Alice"

echo "Hello $name"

read -p "Enter your name: " user

echo "Welcome, $user!"
```

If / Else

```
#!/bin/bash

if [ $disk_usage -gt 80 ]; then

    echo "WARNING: Disk is almost full!"

else

    echo "Disk is fine"

fi
```

Loops

```
#!/bin/bash

for server in "web1" "web2" "db1"; do

    echo "Checking $server..."

    ping -c 1 $server > /dev/null && echo "UP" || echo "DOWN"

done
```

Practical: Backup Script

```
#!/bin/bash

SOURCE="/var/www/html"

DEST="/backup"

DATE=$(date +%Y-%m-%d)
```

```
tar -czf "$DEST/backup-$DATE.tar.gz" "$SOURCE"

echo "Backup done: backup-$DATE.tar.gz"

# Remove backups older than 7 days

find "$DEST" -name "*.tar.gz" -mtime +7 -delete
```

■ **Real World:** At many companies, bash scripts run at midnight to backup databases, clean old log files, and send health check emails.

⇒ ■ PRACTICE EXERCISE

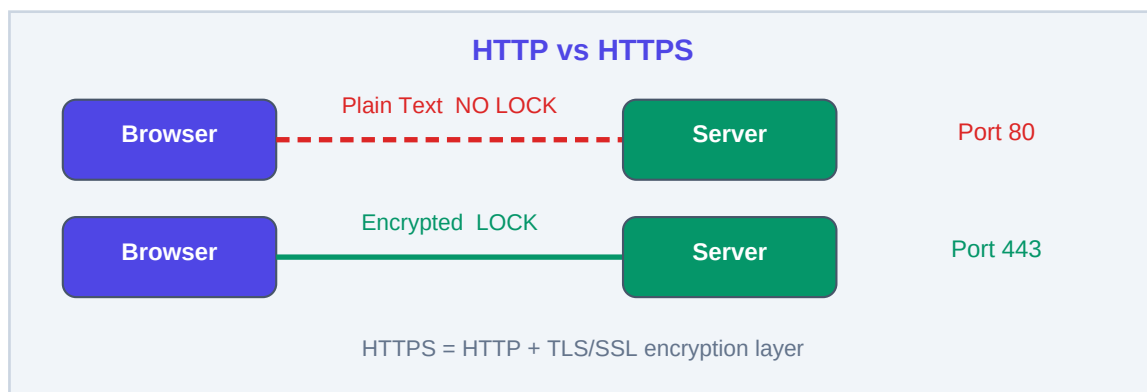
- Write a script that prints your name and current directory
- Write a script with a variable for your city
- Write if/else that checks if a file exists
- Bonus: Loop to create 5 files (file1.txt to file5.txt)

■ Chapter 2: Networking

Networking is how computers talk to each other. In DevOps, you must understand how data travels from a user to your server and back.

2.1 HTTP vs HTTPS

HTTP is the language browsers and servers use to talk. **HTTPS** is the same language but with **encryption** — nobody can spy on your data.



- **HTTP** — data travels as plain text. Anyone can read it.
- **HTTPS** — data is encrypted. Only you and the server can read it.
- **HTTP** uses port **80**, **HTTPS** uses port **443**

How HTTPS Works

1. Browser connects to server
2. Server sends its SSL certificate (like an ID card)
3. Browser checks the certificate is valid
4. An encryption key is created together
5. All data flows encrypted
6. You see the  lock icon

```
curl -I https://example.com          # check HTTP headers
openssl s_client -connect site.com:443 # check SSL cert
```

■ **Real World:** Online banking always uses **HTTPS**. Without it, your password would travel as plain text that anyone could read!

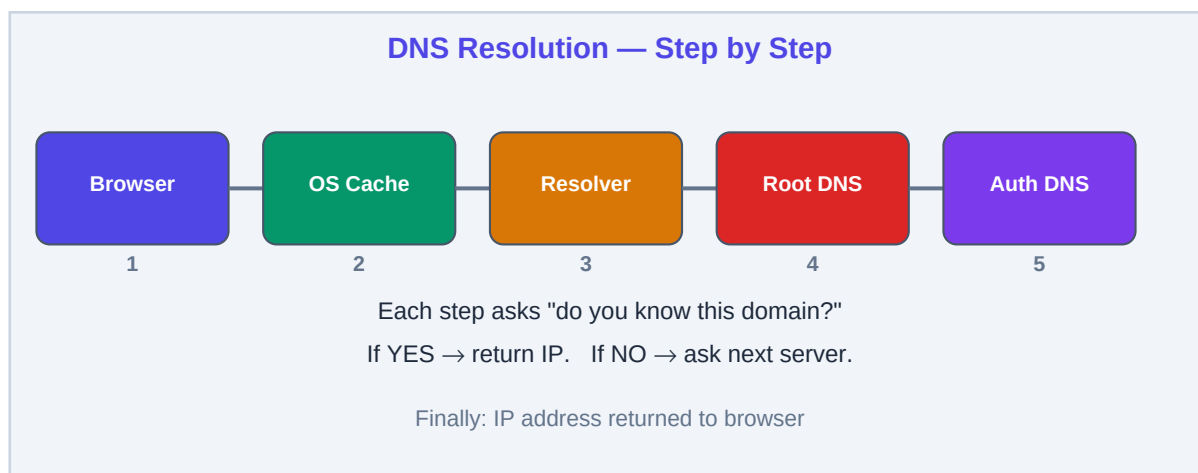
⇒ ■ PRACTICE EXERCISE

- Visit `http://example.com` — does it redirect to `https://`?
- Click the lock icon on any website to see the certificate

- Run: `curl -I https://google.com` — look for status code
- Run: `curl -I http://google.com` — notice redirect to HTTPS

2.2 How DNS Works — Step by Step

DNS is the phone book of the internet. It converts a name like **google.com** into an IP address like **142.250.190.14**. Computers use IP addresses; humans prefer names.



Step by Step

- **Step 1 — Browser Cache:** "Did I look up google.com recently? I remember it!"
- **Step 2 — OS Cache:** Check computer memory for saved results
- **Step 3 — Resolver DNS:** Ask your internet provider's DNS server
- **Step 4 — Root Server:** "I don't know, but I know who handles .com"
- **Step 5 — Authoritative DNS:** "google.com = 142.250.190.14 — here you go!"

```
nslookup google.com      # simple DNS lookup
dig google.com           # detailed DNS lookup
dig +trace google.com    # see full DNS path
cat /etc/resolv.conf     # your DNS servers
```

■ **Note:** TTL (Time To Live) = how long DNS servers remember an answer. Low TTL = changes spread faster but DNS is slower.

■ **Real World:** *Company moves to new server and updates DNS. For a few hours, some users still reach the old server — their DNS cache has not expired yet. This is called DNS propagation.*

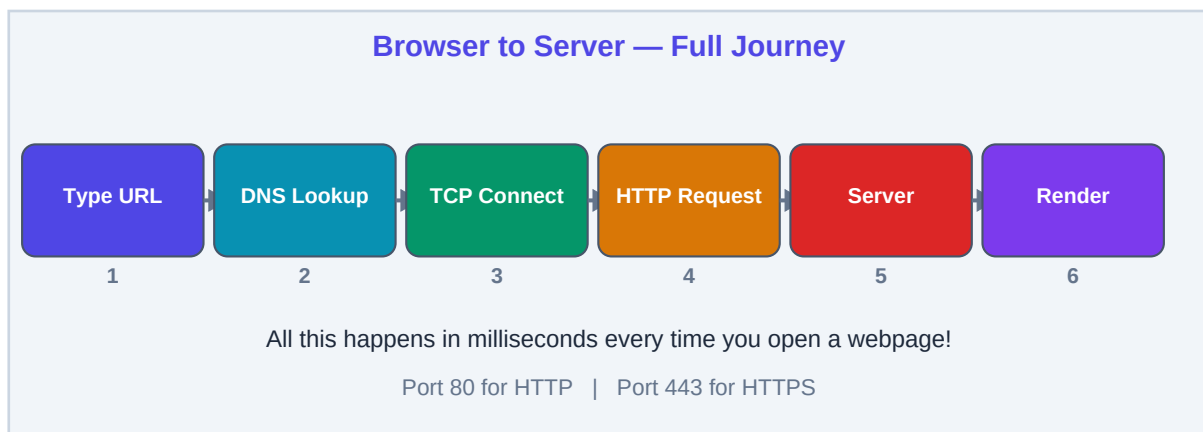
⇒ ■ PRACTICE EXERCISE

- Run: `nslookup google.com` — note the IP

- Run: `nslookup github.com` — different IP!
- Run: `dig google.com` — more details
- Run: `cat /etc/resolv.conf` — see your DNS server

2.3 How a Request Goes from Browser to Server

When you press Enter after typing a URL, a lot happens in under 1 second. Let's follow the full journey.



- **1. Type URL:** "https://myapp.com/login"
- **2. DNS Lookup:** Find the IP for myapp.com
- **3. TCP Connect:** 3-way handshake with the server
- **4. TLS Handshake:** Create an encrypted tunnel (HTTPS)
- **5. HTTP Request:** Browser sends "GET /login HTTP/1.1"
- **6. Server responds:** Returns HTML with status 200 OK
- **7. Browser renders:** Shows you the login page

■ TCP 3-Way Handshake

1. Client → Server: "SYN" (Hello, can we talk?)
2. Server → Client: "SYN-ACK" (Yes! I am ready)
3. Client → Server: "ACK" (Great, let's go!)

```
curl -v https://httpbin.org/get    # see full request/response
traceroute google.com              # see every hop your data takes
```

—■ PRACTICE EXERCISE

- Run: `curl -v https://httpbin.org/get` — read each step
- Open browser DevTools (F12) → Network tab → refresh page
- Run: `netstat -an | grep ESTABLISHED` — see active connections

2.4 Ports and Firewalls

A **port** is like a door into your server. Different services use different doors. A **firewall** controls which doors are open and which are locked.

Port	Service — Used for
22	SSH — Secure remote login
80	HTTP — Normal web traffic
443	HTTPS — Encrypted web traffic
3306	MySQL — Database
5432	PostgreSQL — Database
6379	Redis — Cache server
8080	HTTP Alt — Dev web server
3000	Node.js — Default port
5000	Flask — Python web apps

UFW Firewall (Ubuntu)

```
sudo ufw status          # check firewall status
sudo ufw enable          # turn on firewall
sudo ufw allow 22        # ALWAYS allow SSH first!
sudo ufw allow 80        # allow web traffic
sudo ufw allow 443       # allow HTTPS
sudo ufw allow "Nginx Full" # allow Nginx
sudo ufw deny 3306       # block MySQL from outside
sudo ufw status numbered # see all rules
ss -tulnp               # see all open ports
```

■ **Note:** ALWAYS allow port 22 before enabling the firewall — otherwise you will lock yourself out of the server!

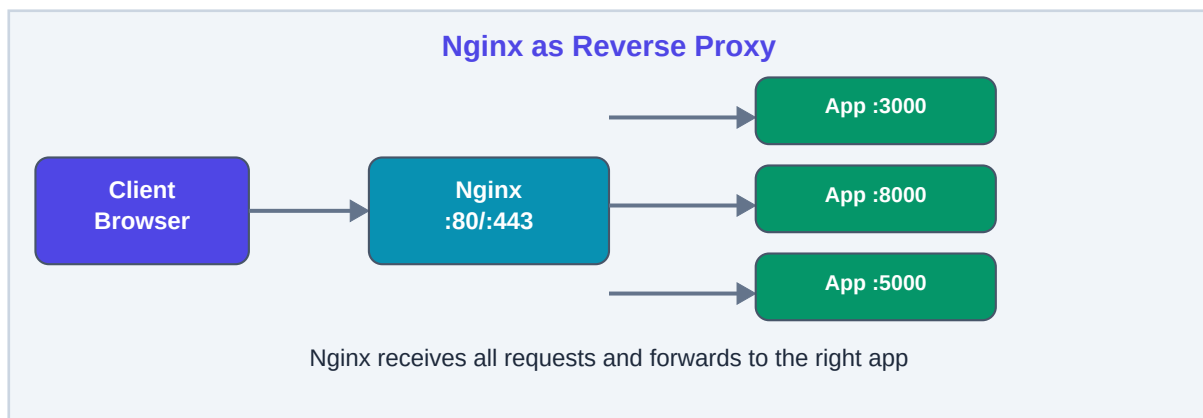
■ **Real World:** MySQL is on port 3306. You only want your app server to connect to it. Block 3306 from outside: `"ufw deny 3306"`.

■ PRACTICE EXERCISE

- Run: `ss -tulnp` — see listening ports
- Run: `sudo ufw status`
- Allow port 8080: `sudo ufw allow 8080`
- Remove the rule: `sudo ufw delete allow 8080`

2.5 Reverse Proxy using Nginx

Nginx is a very fast web server. As a **reverse proxy**, it sits in front of your app and forwards requests to it. Think of it as a receptionist who sends visitors to the right room.



- Your app can run on port 3000 but users connect on port 80
- Nginx handles SSL/HTTPS — your app does not need to worry about it
- Can run multiple apps on one server
- Easy caching, rate limiting, logging

Install Nginx

```
sudo apt update && sudo apt install nginx -y
sudo systemctl start nginx
sudo systemctl enable nginx
sudo systemctl status nginx
sudo nginx -t                # test config
```

Reverse Proxy Config

```
# /etc/nginx/sites-available/myapp
```

```
server {  
    listen 80;  
  
    server_name myapp.com;  
  
    location / {  
        proxy_pass http://localhost:3000;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
    }  
}
```

```
# Enable the site  
  
sudo ln -s /etc/nginx/sites-available/myapp /etc/nginx/sites-enabled/  
  
sudo nginx -t          # test first!  
  
sudo systemctl reload nginx  # reload – no downtime
```

■ **Tip:** Always run "sudo nginx -t" before reloading. It checks for typos.

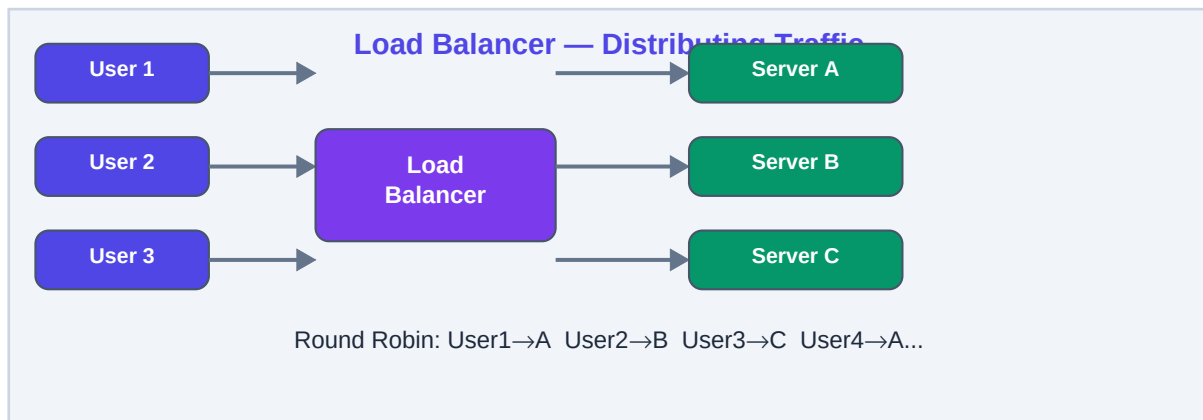
■ **Real World:** *Node.js app runs on port 3000. Users visit <http://mysite.com> (port 80). Nginx receives on port 80 and forwards to port 3000 silently.*

== ■ PRACTICE EXERCISE

- Install: `sudo apt install nginx -y`
- Open browser: `http://localhost` — see Nginx welcome page
- Check logs: `tail -f /var/log/nginx/access.log`
- Reload: `sudo systemctl reload nginx`

2.6 Load Balancer Concept

Imagine 1000 people visiting your website at the same time. One server cannot handle all that! A **load balancer** splits traffic across multiple servers so no one gets overloaded.



■ Bank Queue Analogy

A bank with 3 counters. A manager at the door (load balancer) sends each customer to the counter with the shortest queue. No counter gets too busy. Everyone is served fast.

Nginx as Load Balancer

```
# /etc/nginx/sites-available/loadbalancer

upstream myapp_servers {
    server 10.0.0.1:3000; # Server A
    server 10.0.0.2:3000; # Server B
    server 10.0.0.3:3000; # Server C
}

server {
    listen 80;
    location / {
        proxy_pass http://myapp_servers;
    }
}
```

- **Round Robin (default):** Request 1→A, 2→B, 3→C, 4→A... (equal sharing)
- **Least Connections:** Send to the server handling fewest requests
- **IP Hash:** Same user always goes to same server (good for sessions)

■ **Real World:** Netflix, Amazon, and every large website uses load balancers. Your request might go to any one of thousands of servers — you never notice!

— ■ PRACTICE EXERCISE

- Read nginx config: `cat /etc/nginx/nginx.conf`
- Understand the "upstream" block
- Think: if Server A goes down, what happens?
- Answer: Nginx automatically sends traffic to B and C

■ Chapter 3: Git

Git tracks every change in your code. It lets a whole team work on the same project without overwriting each other's work. Think of it as "save history" for your codebase.

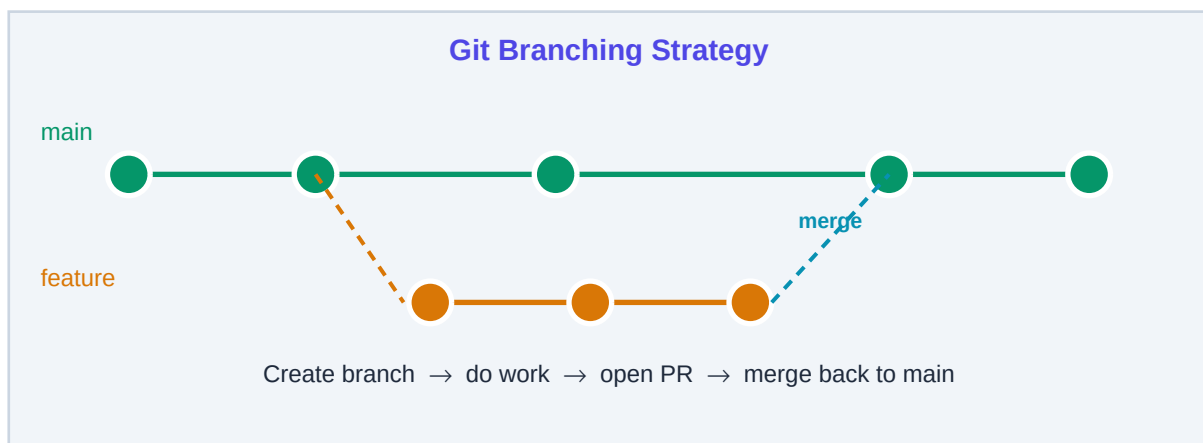
Git Basics

```
git config --global user.name "Alice Smith"
git config --global user.email "alice@company.com"

git init                                # start new git project
git clone https://github.com/user/project.git # clone
git status                              # see what changed
git add .                               # stage all changes
git commit -m "Add login"               # save a snapshot
git push origin main                    # push to GitHub
git pull origin main                    # get latest from GitHub
```

3.1 Branching Strategy

A **branch** is a separate copy of code where you work freely. **main** is the official, production-ready code. Never work directly on main — always use a branch!



■ The Golden Rule of Branching

main = always working, always deployable code

feature branches = where you do ALL your work

Never commit directly to main!

Branch Commands

```
git branch                # see all branches
git checkout -b feature/login-page # create + switch
git branch --show-current   # which branch am I on?
git push origin feature/login-page # push branch to GitHub
git branch -d feature/login-page  # delete after merging
```

- **feature/login-page** — adding a new feature
- **fix/crash-on-login** — fixing a bug
- **hotfix/security-patch** — urgent production fix

■ **Real World:** You work on "feature/user-profile". Your colleague works on "feature/payment-system". Both independent. No conflicts. When done, both merge into main.

— ■ PRACTICE EXERCISE

- git init in a new folder
- Create hello.txt, add and commit
- Create branch: git checkout -b feature/test
- Edit hello.txt, commit again
- Switch back: git checkout main — notice hello.txt is unchanged!

3.2 Pull Request Flow

A **Pull Request (PR)** says: "I finished my work. Please review it and merge it into main."

Step	Action
1	Create branch for your task
2	Write code and commit
3	Push branch to GitHub
4	Open a Pull Request on GitHub
5	Team reviews the code
6	Leave comments, request changes
7	Developer fixes feedback
8	Reviewer approves

9	Merge PR into main
10	Delete the feature branch

PR Commands

```
git checkout -b feature/add-search  
  
git add . && git commit -m "Add search to header"  
  
git push origin feature/add-search  
  
# Go to GitHub → click "New Pull Request"  
  
# base: main ← compare: feature/add-search  
  
# After merge, clean up:  
  
git checkout main && git pull origin main  
  
git branch -d feature/add-search
```

■ **Tip:** Write a good PR description: What changed? Why? How to test it?

■ **Note:** Never merge your own PR without at least one reviewer — that's the whole point!

— ■ PRACTICE EXERCISE

- Create a GitHub account
- Fork any public repo
- Clone it, create a branch, make a small change
- Push and open a Pull Request
- Read the GitHub PR diff view

3.3 Merge Conflicts (With Example)

A **merge conflict** happens when two people change the **same line** in the same file. Git does not know which version to keep, so it asks you to decide. Don't panic — conflicts are normal!

How a Conflict Looks

Original file (main):

```
greeting = "Hello, World!"
```

Alice changed it:

```
greeting = "Hello, Alice!"
```

Bob changed it (different text, same line):

```
greeting = "Hello, Bob!"
```

When Bob merges, Git shows:

```
<<<<<< HEAD (your branch)
greeting = "Hello, Bob!"
=====
greeting = "Hello, Alice!"
>>>>>> feature/alice-branch
```

■ Reading Conflict Markers

```
<<<<<< HEAD = YOUR changes (current branch)

===== = divider between the two versions

>>>>>> other = THEIR changes (incoming branch)
```

Fixing the Conflict

Edit the file, pick (or combine) the versions, then remove all markers:

```
# Option A: keep yours
greeting = "Hello, Bob!"

# Option B: keep theirs
greeting = "Hello, Alice!"

# Option C: combine (most common)
greeting = "Hello, Alice and Bob!"
```

```
git add filename.py          # mark as resolved
git commit -m "Resolve conflict" # complete the merge
git merge --abort             # cancel merge if needed
```

■ **Tip:** VS Code shows conflicts with colour and buttons: "Accept Current", "Accept Incoming", or "Accept Both".

How to Avoid Conflicts

- Pull from main regularly: `git pull origin main` on your branch
- Make small, focused commits — big branches = more conflicts
- Communicate with team about who works on which files

- Merge your branch back to main quickly

—■ PRACTICE EXERCISE

- Create repo with greeting.txt
- Create branch-A and change line 1
- Create branch-B from main, also change line 1 (different text)
- Merge branch-A: `git merge branch-A`
- Merge branch-B — you will see a conflict!
- Fix it, then: `git add . && git commit -m "fix conflict"`

■ ■ Mini Project: Deploy a Simple App

■ Project Goal

Deploy a simple Python web app on Ubuntu using Linux + Nginx. This uses everything you learned in this guide!

You need: Ubuntu server (or VirtualBox/WSL), Python 3, Nginx

Step 1 — Set Up the Server

```
sudo apt update && sudo apt upgrade -y  
sudo apt install python3 python3-pip nginx -y  
mkdir -p ~/myapp && cd ~/myapp
```

Step 2 — Create the Python App

```
pip3 install flask gunicorn  
nano app.py
```

```
# File: app.py  
from flask import Flask  
app = Flask(__name__)  
  
@app.route("/")  
def home():  
    return "<h1>Hello from My DevOps App!</h1>"  
  
@app.route("/health")  
def health():  
    return {"status": "ok"}  
  
if __name__ == "__main__":  
    app.run(host="0.0.0.0", port=5000)
```

```
python3 app.py    # test: visit http://localhost:5000
```

Step 3 — Run as Background Service


```
sudo nano /etc/systemd/system/myapp.service
```

```
[Unit]

Description=My Flask App
After=network.target


[Service]

User=ubuntu

WorkingDirectory=/home/ubuntu/myapp

ExecStart=/usr/local/bin/gunicorn -w 2 -b 127.0.0.1:5000 app:app

Restart=always


[Install]

WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable myapp
sudo systemctl start myapp
sudo systemctl status myapp    # check it runs
sudo journalctl -u myapp -f    # watch logs
```

Step 4 — Configure Nginx

```
sudo nano /etc/nginx/sites-available/myapp
```

```
server {

    listen 80;

    server_name _;

    location / {

        proxy_pass http://127.0.0.1:5000;

        proxy_set_header Host $host;

        proxy_set_header X-Real-IP $remote_addr;

    }

}
```

```
sudo ln -s /etc/nginx/sites-available/myapp /etc/nginx/sites-enabled/
sudo rm /etc/nginx/sites-enabled/default
```

```
sudo nginx -t          # test config
sudo systemctl reload nginx  # reload
```

Step 5 — Set Up Firewall

```
sudo ufw allow 22    # SSH first!
sudo ufw allow 80
sudo ufw allow 443
sudo ufw enable
sudo ufw status
```

Step 6 — Test Everything

```
curl http://localhost          # should see HTML
curl http://localhost/health    # should see {"status":"ok"}
tail -f /var/log/nginx/access.log # watch requests
sudo journalctl -u myapp -n 50   # see app logs
```

■ Congratulations! You Just Deployed an App!

You used: Linux commands, Nginx reverse proxy, systemd service, firewall rules, and log monitoring — all from this guide!

Next: Add HTTPS with Certbot, set up Git hooks for auto-deploy, add monitoring with Prometheus + Grafana.

■ Quick Reference Cheatsheet

Command	What it does
<code>chmod 755 file</code>	Owner all rights, others read+exec
<code>ls -la</code>	List all files with permissions
<code>ps aux grep app</code>	Find a running process
<code>kill -9 <PID></code>	Force stop a process
<code>tail -f /var/log/app.log</code>	Watch logs live
<code>ssh user@ip</code>	Connect to remote server
<code>scp file user@ip:/path/</code>	Copy file to server
<code>curl -I https://site.com</code>	Check HTTP headers
<code>nslookup domain.com</code>	DNS lookup
<code>ss -tulnp</code>	See all open ports
<code>sudo ufw allow 80</code>	Open port 80 in firewall
<code>sudo nginx -t</code>	Test Nginx config
<code>sudo systemctl reload nginx</code>	Reload Nginx (no downtime)
<code>git checkout -b feature/x</code>	Create and switch to branch
<code>git add . && git commit -m</code>	Stage and commit changes
<code>git push origin branch</code>	Push branch to GitHub
<code>git pull origin main</code>	Get latest from main
<code>git log --oneline</code>	See commit history
<code>git merge --abort</code>	Cancel a merge with conflicts

■ What to Learn Next

- **Docker** — Package your app into containers — run anywhere
- **CI/CD** — Automate testing and deployment (GitHub Actions)
- **Kubernetes** — Manage hundreds of containers automatically
- **AWS / GCP** — Deploy on real cloud servers
- **Terraform** — Create infrastructure with code (IaC)
- **Monitoring** — Track app health with Prometheus + Grafana

DevOps Foundation Guide • Author: Er. Meet Kanteliya • Keep learning! ■